

AIKD-2026-PROJEKT

Dokumentacja techniczna

Anton Filip Bartek

10 czerwca 2026

Dokumentacja modułów projektu algorytmicznego obejmującego przydział pracy, geometrię obliczeniową, zapytania przedziałowe, kompresję danych oraz wizualizację.

Spis treści

1	Podział pracy	5
1.1	Anton	5
1.2	Filip	5
1.3	Bartek	5
1.4	Praca wspólna	6
2	Opis ogólny projektu	7
3	Moduł <code>main.py</code>	8
3.1	Opis	8
3.2	Etapy działania programu	8
3.2.1	Generowanie krasnoludków	8
3.2.2	Generowanie kopalni	9
3.2.3	Przydział pracy	9
3.2.4	Wyznaczanie trasy patrolu	9
3.2.5	Rozstawienie strażników	10
3.2.6	Obsługa ataków	10
3.2.7	Raport i archiwizacja	10
3.3	Struktura modułu	10
3.3.1	<code>wizualizacja(...)</code>	10
3.3.2	<code>generuj_raport_dzienny(...)</code>	11
3.3.3	<code>main(...)</code>	11
3.3.4	<code>startup()</code>	12
4	Moduł <code>geometry.py</code>	12
4.1	Opis	12
4.2	Iloczyn wektorowy	13
4.3	Graham Scan	13
4.4	Obliczanie obwodu	13
4.5	Struktura modułu	14

5	Moduł <code>range_query.py</code>	14
5.1	Opis	14
5.2	Sparse Table	14
5.3	Struktura modułu	15
5.3.1	<code>SparseTable</code>	15
5.3.2	<code>__init__(arr)</code>	15
5.3.3	<code>query(l, r)</code>	15
6	Moduł <code>archive.py</code>	15
6.1	Opis	15
6.2	Algorytm Huffmana	16
6.3	Struktura Huffmana	16
6.3.1	<code>Node</code>	16
6.3.2	<code>Huffman</code>	17
6.3.3	<code>kompresuj()</code>	17
6.3.4	<code>dekompresuj(kod=None)</code>	17
6.4	Algorytm KMP	17
6.5	Struktura KMP	18
6.5.1	<code>KMP._stworz_LPS(worzec)</code>	18
6.5.2	<code>KMP.algorytm_KMP(tekst, wzorzec)</code>	18
7	Część C++	18
7.1	Opis	18
7.2	Plik <code>models2.cpp</code>	18
7.2.1	<code>Node</code>	19
7.2.2	<code>Worker</code>	19
7.2.3	<code>Mine</code>	19
7.3	Plik <code>algorithm.cpp</code>	20
7.3.1	<code>Edge</code>	20
7.3.2	<code>Graph</code>	20
7.3.3	<code>addEdge(from, to, cap, cost)</code>	21
7.3.4	<code>processGraph(sourceIdx, sinkIdx)</code>	21

7.3.5	<code>getAssignments(dwarves, mines)</code>	21
7.3.6	<code>minimumCostMaximumFlow(workers, mines)</code>	21
7.4	Plik <code>api.cpp</code>	22
7.4.1	Surowiec	22
7.4.2	Punkt	22
7.4.3	Kopalnia	23
7.4.4	Krasnoludek	23
7.4.5	<code>mcmf</code>	23
8	Moduł <code>gen_test.py</code>	23
8.1	Opis	23
8.2	Działanie modułu	24
8.3	Eksport wyników do Excela	25
8.4	Format danych w arkuszu	25
8.5	Struktura modułu	26
8.5.1	<code>ILE_TESTOW</code>	26
8.5.2	<code>NAZWA_PLIKU</code>	26
8.5.3	<code>test(krasnale, miejsce, ataki)</code>	26
8.5.4	Obsługa pojedynczej wartości zakresu	27
8.6	Wymagania modułu	27
9	Pliki generowane przez program	27
9.1	<code>zadanie_out.txt</code>	27
9.2	<code>czasowe.xlsx</code>	27
10	Wymagania	28
11	Złożoność obliczeniowa	28
12	Oznaczenia	29
13	Uwagi implementacyjne	29

1 Podział pracy

1.1 Anton

Anton odpowiadał za całą część napisaną w języku C++ oraz za plik `models.pyi`.

Zakres pracy:

- implementacja struktur danych po stronie C++,
- implementacja algorytmu maksymalnego przepływu o minimalnym koszcie,
- przygotowanie integracji C++ z Pythonem przez `pybind11`,
- przygotowanie interfejsu typów dla Pythona w pliku `models.pyi`.

Pliki:

- `algorithm.cpp`,
- `api.cpp`,
- `models2.cpp`,
- `models.pyi`.

1.2 Filip

Filip odpowiadał za moduły związane z geometrią oraz zapytaniami przedziałowymi.

Zakres pracy:

- wyznaczanie otoczki wypukłej,
- obliczanie obwodu otoczki,
- implementacja struktury Sparse Table,
- obsługa zapytań RMQ.

Pliki:

- `geometry.py`,
- `range_query.py`.

1.3 Bartek

Bartek odpowiadał za moduły związane z archiwizacją danych oraz głównym przebiegiem programu.

Zakres pracy:

- implementacja algorytmu Huffmana,
- implementacja algorytmu KMP,
- generowanie raportu dziennego,
- główna logika programu,
- wizualizacja działania programu,
- pomiar czasu działania.

Pliki:

- `archive.py`,
- `main.py`.

1.4 Praca wspólna

Cały zespół pracował nad modulem testowym służącym do wykonywania wielu testów czasowych.

Plik:

- `gen_test.py`.

2 Opis ogólny projektu

Projekt symuluje działanie systemu zarządzania krasnoludkami i kopalniami.

Program:

- generuje krasnoludki oraz kopalnie,
- przydziela krasnoludki do kopalni zgodnie z preferencjami i odległością,
- wyznacza trasę patrolu wokół aktywnych kopalń,
- rozmieszcza strażników na trasie,
- obsługuje ataki na wybrane odcinki granicy,
- generuje raport dzienny,
- kompresuje raport algorytmem Huffmana,
- wyszukuje wzorce w raporcie algorytmem KMP,
- umożliwia uruchomienie wizualizacji graficznej,
- umożliwia wykonywanie automatycznych testów czasowych.

Projekt łączy kod Pythona z kodem C++. Część C++ odpowiada za wydajny algorytm przydziału pracy, natomiast Python obsługuje logikę programu, geometrię, archiwizację, testy oraz wizualizację.

3 Moduł `main.py`

3.1 Opis

Moduł `main.py` odpowiada za główny przebieg programu.

Łączy on wszystkie najważniejsze części projektu:

- generowanie krasnoludków,
- generowanie kopalni,
- przydzielanie krasnoludków do kopalni,
- wyznaczanie trasy patrolu,
- obsługę ataków,
- generowanie raportu,
- kompresję raportu,
- wyszukiwanie wzorca w raporcie,
- opcjonalną wizualizację.

Program może działać w dwóch trybach:

- tryb tekstowy,
- tryb graficzny z wykorzystaniem biblioteki `pygame`.

3.2 Etapy działania programu

3.2.1 Generowanie krasnoludków

Program generuje zadaną liczbę krasnoludków.

Każdy krasnoludek otrzymuje:

- losową pozycję,
- losowo wybrany preferowany typ surowca.

Dostępne typy surowców:

- złoto,
- węgiel,
- miedź,
- uran.

Złożoność:

$$O(K)$$

gdzie K oznacza liczbę krasnoludków.

3.2.2 Generowanie kopalni

Na podstawie liczby krasnoludków preferujących dany surowiec program tworzy odpowiednią liczbę kopalni.

Liczba kopalni zależy od:

- liczby krasnoludków danego typu,
- liczby miejsc dostępnych w jednej kopalni.

Złożoność:

$$O(M)$$

gdzie M oznacza liczbę kopalni.

3.2.3 Przydział pracy

Po wygenerowaniu danych program wywołuje funkcję:

```
models.mcmf(krasnale, kopalnie)
```

Funkcja ta korzysta z implementacji C++ i odpowiada za przypisanie krasnoludków do kopalni.

Przydział uwzględnia:

- preferowany typ surowca krasnoludka,
- typ surowca kopalni,
- odległość krasnoludka od kopalni,
- maksymalną pojemność kopalni.

W projekcie wykorzystywany jest algorytm maksymalnego przepływu o minimalnym koszcie.

3.2.4 Wyznaczanie trasy patrolu

Po przydzieleniu krasnoludków wybierane są aktywne kopalnie, czyli takie, do których przypisano co najmniej jednego pracownika.

Następnie program:

1. wyznacza otoczkę wypukłą aktywnych kopalń,
2. oblicza obwód otoczki,
3. traktuje obwód jako długość trasy patrolu księcia.

Wykorzystywane funkcje:

- `graham_scan(uzyte_kopalnie)`,
- `perimeter(punkty_otoczki)`.

Złożoność:

$$O(M \log M)$$

3.2.5 Rozstawienie strażników

Na podstawie długości trasy patrolu program oblicza liczbę strażników.

Strażnicy są rozmieszczani co 10 metrów trasy.

Minimalna liczba strażników wynosi 2.

3.2.6 Obsługa ataków

Program losuje przedziały strażników objęte atakiem.

Dla każdego ataku wybierany jest najgłośniejszy strażnik z danego przedziału. Do tego celu używana jest struktura Sparse Table.

Złożoność pojedynczego zapytania:

$$O(1)$$

3.2.7 Raport i archiwizacja

Program generuje raport dzienny, a następnie kompresuje go algorytmem Huffmana.

Skompresowany raport zapisywany jest do pliku:

`zadanie_out.txt`

Po kompresji wykonywana jest dekompresja kontrolna, która sprawdza poprawność algorytmu.

3.3 Struktura modułu

3.3.1 wizualizacja(...)

Funkcja odpowiada za graficzne przedstawienie działania programu.

Pokazuje:

- kopalnie,
- krasnoludki,
- przemieszczanie krasnoludków do kopalni,
- trasę patrolu,
- strażników,
- ataki,
- najgłośniejszego strażnika,
- wykres kompresji raportu.

3.3.2 `generuj_raport_dzienny(...)`

Tworzy raport tekstowy z działania programu.

Raport zawiera:

- datę utworzenia,
- liczbę krasnoludków,
- liczbę aktywnych kopalni,
- przykładowe przydziały,
- długość trasy patrolu,
- liczbę punktów otoczki,
- liczbę strażników,
- liczbę ataków.

Zwraca:

- raport jako wartość typu `str`.

3.3.3 `main(...)`

Główna funkcja programu.

Wykonuje:

1. konfigurację logowania,
2. generowanie danych,
3. przydział krasnoludków do kopalni,
4. wyznaczanie trasy patrolu,

5. przygotowanie Sparse Table,
6. obsługę ataków,
7. generowanie raportu,
8. kompresję Huffmana,
9. wyszukiwanie KMP,
10. opcjonalną wizualizację,
11. pomiar czasu działania.

Zwraca:

- czas działania w trybie bez GUI,
- brak wartości w trybie graficznym.

3.3.4 `startup()`

Funkcja pobiera dane od użytkownika.

Dane wejściowe:

- liczba krasnoludków,
- liczba miejsc w kopalni,
- liczba ataków,
- decyzja o włączeniu GUI,
- decyzja o włączeniu wizualizacji.

4 Moduł `geometry.py`

4.1 Opis

Moduł `geometry.py` odpowiada za operacje geometryczne używane przy wyznaczaniu trasy patrolu.

Główne zadania modułu:

- sprawdzanie orientacji trzech punktów,
- wyznaczanie otoczki wypukłej,
- obliczanie obwodu otoczki.

Moduł wykorzystuje algorytm Graham Scan.

4.2 Iloczyn wektorowy

Funkcja:

`orientation(a, b, c)`

sprawdza orientację trzech punktów.

Możliwe wyniki:

- wartość dodatnia – skręt w lewo,
- wartość ujemna – skręt w prawo,
- zero – punkty współliniowe.

Złożoność:

$O(1)$

4.3 Graham Scan

Funkcja:

`graham_scan(mines)`

wyznacza otoczkę wypukłą dla listy kopaliń.

Etapy działania:

1. Zamiana kopaliń na punkty.
2. Wybór punktu startowego o najmniejszej współrzędnej y .
3. Sortowanie punktów według kąta polarnego.
4. Budowanie stosu punktów otoczki.
5. Usuwanie punktów powodujących niewłaściwy skręt.
6. Zwrócenie listy punktów otoczki.

Złożoność:

$O(N \log N)$

4.4 Obliczanie obwodu

Funkcja:

`perimeter(hull)`

oblicza obwód wielokąta utworzonego przez punkty otoczki wypukłej.

Działanie:

- sumuje odległości między kolejnymi punktami,
- uwzględnia odcinek od ostatniego punktu do pierwszego.

Złożoność:

$$O(N)$$

4.5 Struktura modułu

Element	Opis
<code>orientation(a, b, c)</code>	Sprawdza orientację trzech punktów.
<code>graham_scan(mines)</code>	Wyznacza otoczkę wypukłą dla kopaliń.
<code>perimeter(hull)</code>	Oblicza obwód otoczki wypukłej.

5 Moduł `range_query.py`

5.1 Opis

Moduł `range_query.py` odpowiada za szybkie odpowiadanie na zapytania przedziałowe.

W projekcie jest używany do znalezienia najgłośniejszego strażnika na zaatakowanym odcinku granicy.

Moduł zawiera klasę:

`SparseTable`

5.2 Sparse Table

Sparse Table to struktura danych przeznaczona do zapytań przedziałowych na statycznej tablicy.

W projekcie przechowuje ona głośności strażników.

Po przygotowaniu struktury można w czasie stałym znaleźć największą wartość w dowolnym przedziale.

Złożoność przygotowania:

$$O(N \log N)$$

Złożoność zapytania:

$$O(1)$$

5.3 Struktura modułu

5.3.1 SparseTable

Klasa reprezentująca strukturę Sparse Table.

Przechowuje:

- tablicę logarytmów,
- kolejne poziomy Sparse Table,
- pary wartości w postaci (wartość, indeks).

5.3.2 __init__(arr)

Buduje strukturę Sparse Table dla tablicy wejściowej.

Działanie:

1. Oblicza wartości logarytmów.
2. Tworzy poziom zerowy z pojedynczych elementów.
3. Buduje kolejne poziomy dla przedziałów długości 2^k .
4. Zapisuje maksimum oraz indeks maksimum.

Złożoność:

$$O(N \log N)$$

5.3.3 query(l, r)

Zwraca maksimum na przedziale $[l, r]$.

Zwraca:

- największą wartość,
- indeks elementu o największej wartości.

Złożoność:

$$O(1)$$

6 Moduł archive.py

6.1 Opis

Moduł `archive.py` odpowiada za archiwizację raportu dziennego.

Zawiera dwa główne algorytmy:

- algorytm Huffmana,
- algorytm KMP.

Algorytm Huffmana służy do kompresji i dekompresji raportu.

Algorytm KMP służy do wyszukiwania wzorca w raporcie.

6.2 Algorytm Huffmana

Algorytm Huffmana jest algorytmem kompresji bezstratnej.

Znaki występujące częściej otrzymują krótsze kody binarne. Znaki występujące rzadziej otrzymują dłuższe kody binarne.

Etapy działania:

1. Zliczenie wystąpień znaków.
2. Utworzenie kolejki priorytetowej.
3. Budowa drzewa Huffmana.
4. Wygenerowanie kodów binarnych.
5. Zakodowanie tekstu.
6. Odtworzenie tekstu przez dekompresję.

Złożoność:

$$O(R + A \log A)$$

gdzie:

- R – długość tekstu,
- A – liczba różnych znaków.

6.3 Struktura Huffmana

6.3.1 Node

Klasa reprezentująca węzeł drzewa Huffmana.

Przechowuje:

- znak,
- częstotliwość występowania,
- lewe dziecko,
- prawe dziecko.

6.3.2 Huffman

Klasa odpowiadająca za:

- przygotowanie drzewa Huffmana,
- wygenerowanie kodów,
- kompresję tekstu,
- dekompresję tekstu.

6.3.3 `kompresuj()`

Kompresuje tekst do postaci ciągu bitów.

Zwraca:

- skompresowany tekst jako `str`.

6.3.4 `dekompresuj(kod=None)`

Odtwarza tekst ze skompresowanej postaci.

Jeżeli nie zostanie podany argument `kod`, funkcja używa kodu zapisanego wewnątrz obiektu.

Zwraca:

- odtworzony tekst.

6.4 Algorytm KMP

KMP, czyli algorytm Knutha-Morrisa-Pratta, służy do wyszukiwania wzorca w tekście.

Jego zaletą jest to, że po niezgodności znaków nie cofa się po tekście wejściowym.

Wykorzystuje tablicę LPS, która określa najdłuższy właściwy prefiks będący jednocześnie sufiksem.

Etapy działania:

1. Przygotowanie tablicy LPS dla wzorca.
2. Przejście po tekście.
3. Porównywanie znaków tekstu i wzorca.
4. Zapisywanie indeksów wystąpień wzorca.
5. Przesuwanie wzorca na podstawie tablicy LPS.

Złożoność:

$$O(R + W)$$

gdzie:

- R – długość tekstu,
- W – długość wzorca.

6.5 Struktura KMP

6.5.1 `KMP._stworz_LPS(worzec)`

Tworzy tablicę LPS dla wzorca.

Zwraca:

- listę wartości LPS.

6.5.2 `KMP.algorytm_KMP(tekst, wzorzec)`

Wyszukuje wszystkie wystąpienia wzorca w tekście.

Zwraca:

- listę indeksów, od których zaczyna się wzorzec.

7 Część C++

7.1 Opis

Część C++ odpowiada za wydajną implementację struktur danych oraz algorytmu przydziału pracy.

Kod C++ jest udostępniany Pythonowi jako moduł:

`models`

Do integracji wykorzystano bibliotekę:

`pybind11`

7.2 Plik `models2.cpp`

Plik `models2.cpp` zawiera implementację podstawowych struktur danych używanych przez program.

Struktury:

- Node,
- Worker,
- Mine.

7.2.1 Node

Reprezentuje punkt na płaszczyźnie.

Atrybuty:

- x,
- y.

Metody:

- `getDistance(other)` – oblicza odległość euklidesową do innego punktu.

Złożoność:

$$O(1)$$

7.2.2 Worker

Reprezentuje krasnoludka.

Dziedziczy po klasie `Node`.

Przechowuje:

- pozycję krasnoludka,
- preferowany typ surowca.

Metody:

- `getPreference()` – zwraca preferowany surowiec.

Złożoność:

$$O(1)$$

7.2.3 Mine

Reprezentuje kopalnię.

Dziedziczy po klasie `Node`.

Przechowuje:

- pozycję kopalni,
- typ surowca,

- pojemność kopalni.

Metody:

- `getResource()` – zwraca typ surowca,
- `getCapacity()` – zwraca liczbę miejsc w kopalni.

Złożoność:

$$O(1)$$

7.3 Plik `algorithm.cpp`

Plik `algorithm.cpp` zawiera implementację algorytmu maksymalnego przepływu o minimalnym koszcie.

Algorytm służy do przypisywania krasnoludków do kopalń.

W grafie:

- źródło połączone jest z krasnoludkami,
- krasnoludki połączone są z kopalniami zgodnymi z ich preferencją,
- kopalnie połączone są z ujściem,
- koszt krawędzi odpowiada odległości krasnoludka od kopalni,
- pojemność kopalni ogranicza liczbę przypisanych krasnoludków.

7.3.1 Edge

Struktura reprezentująca krawędź w grafie przepływowym.

Przechowuje:

- wierzchołek docelowy,
- pojemność,
- koszt,
- indeks krawędzi odwrotnej.

7.3.2 Graph

Klasa reprezentująca graf przepływowy.

Przechowuje:

- liczbę wierzchołków,
- listę sąsiedztwa.

7.3.3 `addEdge(from, to, cap, cost)`

Dodaje krawędź do grafu.

Funkcja tworzy:

- krawędź właściwą,
- krawędź odwrotną potrzebną do cofania przepływu.

Złożoność:

$$O(1)$$

7.3.4 `processGraph(sourceIdx, sinkIdx)`

Wykonuje właściwe przetwarzanie grafu przepływowego.

Działanie:

1. Szuka najtańszej ścieżki ze źródła do ujścia.
2. Wykorzystuje relaksację krawędzi podobną do algorytmu Bellmana-Forda.
3. Odtwarza znalezioną ścieżkę.
4. Przepycha możliwy przepływ.
5. Aktualizuje pojemności krawędzi.
6. Powtarza działanie, dopóki istnieje ścieżka do ujścia.

7.3.5 `getAssignments(dwarves, mines)`

Po zakończeniu działania algorytmu odczytuje przydziały krasnoludków do kopalni.

Zwraca:

- listę par `(id_krasnoludka, id_kopalni)`.

7.3.6 `minimumCostMaximumFlow(workers, mines)`

Główna funkcja przydzielająca krasnoludków do kopalń.

Działanie:

1. Tworzy graf przepływowy.
2. Dodaje krawędzie ze źródła do krasnoludków.
3. Dodaje krawędzie z krasnoludków do pasujących kopalni.
4. Dodaje krawędzie z kopalni do ujścia.
5. Uruchamia algorytm przepływu.

6. Zwraca listę przydziałów.

Zwraca:

- listę par (krasnodek, kopalnia).

7.4 Plik `api.cpp`

Plik `api.cpp` odpowiada za udostępnienie kodu C++ do Pythona.

Tworzony jest moduł:

`models`

Moduł udostępnia:

- enum `Surowiec`,
- klasę `Punkt`,
- klasę `Kopalnia`,
- klasę `Krasnodek`,
- funkcję `mcmf`.

7.4.1 Surowiec

Enum reprezentujący typ surowca.

Dostępne wartości:

- `ZLOT0`,
- `WEGIEL`,
- `MIEDZ`,
- `URAN`.

7.4.2 Punkt

Pythonowy interfejs dla klasy `Node`.

Udostępnia:

- konstruktor,
- metodę `dystans`,
- pole `x`,
- pole `y`.

7.4.3 Kopalnia

Pythonowy interfejs dla klasy `Mine`.

Udostępnia:

- konstruktor,
- metodę `getSurowiec`.

7.4.4 Krasnoludek

Pythonowy interfejs dla klasy `Worker`.

Udostępnia:

- konstruktor,
- metodę `getSurowiec`.

7.4.5 mcmf

Pythonowy interfejs dla funkcji:

`minimumCostMaximumFlow`

Służy do obliczania przydziału krasnoludków do kopalń.

8 Moduł `gen_test.py`

8.1 Opis

Moduł `gen_test.py` służy do wykonywania wielu testów czasowych programu.

Pozwala uruchamiać funkcję:

`main(...)`

wiele razy dla losowych danych wejściowych.

Dla każdego testu losowane są:

- liczba krasnoludków,
- liczba miejsc w kopalni,
- liczba ataków.

Wyniki testów zapisywane są do pliku Excel:

`czasowe.xlsx`

Jeżeli plik już istnieje, program otwiera go i dopisuje nowe wyniki na końcu arkusza.

8.2 Działanie modułu

Program pobiera od użytkownika trzy zakresy:

- zakres liczby krasnoludków,
- zakres liczby miejsc w kopalni,
- zakres liczby ataków.

Zakres podawany jest jako dwie liczby oddzielone spacją.

Przykład:

100 1000

Jeżeli użytkownik poda tylko jedną liczbę, program traktuje ją jako stałą wartość. W takim przypadku zakres zostaje zamieniony na parę takich samych wartości.

Przykład:

500

zostaje potraktowane tak samo jak:

500 500

Następnie program wykonuje serię testów. Domyślnie wykonywanych jest 40 testów.

Dla każdego testu:

1. losowane są dane wejściowe z podanych zakresów,
2. uruchamiana jest funkcja `main(...)` w trybie bez GUI,
3. pobierany jest czas działania programu,
4. wynik zapisywany jest do listy wyników,
5. wynik testu wypisywany jest w konsoli.

Po zakończeniu wszystkich testów program oblicza średni czas działania.

8.3 Eksport wyników do Excela

Moduł wykorzystuje bibliotekę `openpyxl` do zapisu wyników w pliku Excel.

Jeżeli plik:

```
czasowe.xlsx
```

już istnieje, zostaje otwarty przez funkcję:

```
load_workbook(...)
```

Jeżeli plik nie istnieje, tworzony jest nowy skoroszyt przez:

```
Workbook()
```

Arkusz otrzymuje nazwę:

```
Wyniki
```

Do arkusza dopisywane są:

- nagłówki kolumn,
- wyniki każdego testu,
- pusty wiersz oddzielający dane,
- średni czas wykonania serii testów.

8.4 Format danych w arkuszu

Każdy test zapisywany jest jako osobny wiersz.

Kolumny:

- numer testu,
- liczba krasnoludków,
- liczba miejsc w kopalni,
- liczba ataków,
- czas działania w sekundach.

Czas zapisywany jest z dokładnością do 9 miejsc po przecinku.

Na końcu każdej serii dopisywany jest średni czas działania:

```
Średni czas
```

8.5 Struktura modułu

8.5.1 ILE_TESTOW

Stała określająca liczbę wykonywanych testów.

Domyślna wartość:

40

8.5.2 NAZWA_PLIKU

Stała przechowująca nazwę pliku wynikowego.

Domyślna wartość:

`czasowe.xlsx`

8.5.3 test(krasnale, miejsce, ataki)

Wykonuje serię testów czasowych.

Argumenty:

- `krasnale` – zakres liczby krasnoludków,
- `miejsce` – zakres liczby miejsc w kopalni,
- `ataki` – zakres liczby ataków.

Działanie:

1. Losuje dane wejściowe z podanych zakresów.
2. Uruchamia funkcję `main(...)` w trybie bez GUI.
3. Pobiera zwrócony czas działania.
4. Dodaje wynik do listy wyników.
5. Po zakończeniu testów oblicza średni czas.
6. Otwiera istniejący plik Excel albo tworzy nowy.
7. Dopisuje wyniki do arkusza.
8. Zapisuje plik `czasowe.xlsx`.

Zwraca:

- brak wartości.

8.5.4 Obsługa pojedynczej wartości zakresu

Jeżeli użytkownik poda tylko jedną liczbę dla któregoś parametru, program zamienia ją na zakres jednoelementowy.

Przykład dla liczby krasnoludków:

```
krasnale = (*krasnale, *krasnale)
```

Dzięki temu funkcja:

```
random.randint(*krasnale)
```

może działać zarówno dla zakresu dwóch wartości, jak i dla jednej stałej wartości.

8.6 Wymagania modułu

Moduł korzysta z:

- random,
- main,
- openpyxl.Workbook,
- openpyxl.load_workbook,
- openpyxl.styles.Font,
- pathlib.Path.

9 Pliki generowane przez program

9.1 zadanie_out.txt

Plik generowany przez główny program.

Zawiera skompresowany raport dzienny zapisany w postaci ciągu bitów.

9.2 czasowe.xlsx

Plik generowany przez moduł testowy.

Zawiera wyniki testów czasowych zapisane w arkuszu Excel.

Każda seria testów dopisywana jest do istniejącego pliku, jeżeli plik już istnieje.

Dane zapisywane są w kolumnach:

- numer testu,
- liczba krasnoludków,
- liczba miejsc w kopalni,
- liczba ataków,
- czas działania w sekundach.

Po każdej serii testów dopisywany jest średni czas działania.

10 Wymagania

Program korzysta z:

- Python 3.10+,
- C++,
- pybind11,
- pygame,
- openpyxl,
- pathlib,
- random,
- math,
- logging,
- datetime,
- collections.Counter,
- time.perf_counter,
- heapq,
- typing.

11 Złożoność obliczeniowa

Operacja	Złożoność
Generowanie krasnoludków	$O(K)$
Generowanie kopalni	$O(M)$
Obliczanie odległości	$O(1)$
Dodanie krawędzi w grafie	$O(1)$
Przydział MCMF	zależna od grafu

Orientation	$O(1)$
Sortowanie punktów w Graham Scan	$O(N \log N)$
Graham Scan	$O(N \log N)$
Obliczanie obwodu otoczki	$O(N)$
Budowa Sparse Table	$O(S \log S)$
Zapytanie Sparse Table	$O(1)$
Kompresja Huffmana	$O(R + A \log A)$
Dekompresja Huffmana	$O(B)$
Budowa tablicy LPS	$O(W)$
Wyszukiwanie KMP	$O(R + W)$
Eksport wyników do Excela	$O(T)$
Jeden test czasowy	zależny od parametrów wejściowych

12 Oznaczenia

Symbol	Znaczenie
K	liczba krasnoludków
M	liczba kopalni
N	liczba punktów
S	liczba strażników lub rozmiar tablicy RMQ
R	długość raportu
A	liczba różnych znaków w raporcie
B	długość skompresowanego tekstu
W	długość wzorca
P	liczba punktów ścieżek lub przykładowych przydziałów
T	liczba wykonanych testów czasowych

13 Uwagi implementacyjne

- Główna logika programu znajduje się w pliku `main.py`.
- Część odpowiedzialna za przydział pracy została przeniesiona do C++ dla większej wydajności.
- Integracja C++ z Pythonem odbywa się przez bibliotekę `pybind11`.
- Otoczka wypukła jest wyznaczana tylko dla aktywnych kopalń.
- Sparse Table działa dla danych statycznych, czyli takich, które nie zmieniają się po zbudowaniu struktury.
- Algorytm Huffmana służy do bezstratnej kompresji raportu.
- Poprawność kompresji sprawdzana jest przez dekompresję i porównanie z oryginalnym raportem.

- Algorytm KMP wykorzystywany jest do wyszukiwania sekcji w raporcie.
- Wizualizacja jest opcjonalna i wymaga włączenia trybu GUI.
- Tryb bez GUI może zwracać czas działania, co umożliwia wykonywanie testów czasowych.
- Moduł `gen_test.py` pozwala automatycznie uruchamiać wiele testów z losowymi danymi.
- Wyniki testów są zapisywane do pliku `czasowe.xlsx`.
- Jeżeli plik `czasowe.xlsx` już istnieje, nowe wyniki są dopisywane do istniejącego arkusza.
- Po każdej serii testów obliczany i zapisywany jest średni czas działania.